Governed Data Platform — Technical Architecture Document

Version: 2.0 (Stage 2) **Classification:** Internal — Engineering & Architecture Review **Last Revised:** February 2026

---

1. Executive Summary

The Governed Data Platform is a proof-of-concept for a multi-tenant, role-aware financial data access system built on Databricks Unity Catalog. It demonstrates how row-level security, column-level masking, and audit compliance can be enforced at the database layer rather than in application code — ensuring that even a compromised API or a rogue AI-generated query cannot expose unauthorized data.

The platform serves aggregated financial data (revenue, cost, profit) for five tenant companies through a Flask API and web dashboard. Access is governed by five roles (admin, manager, finance_user, auditor, viewer), each with distinct row visibility and column exposure rules. A natural language query interface powered by Groq AI (Llama 3) allows free-form questions, with every AI-generated SQL statement passing through a five-layer validation firewall before execution.

The architecture is organized into five tiers, described in the sections that follow.

---

2. Architecture Overview

The system is composed of five tiers that operate in sequence for every data request:

Tier	Responsibility
Tier 1 — End Users and Identity	Authentication, role resolution, session context
Tier 2 — API Gateway and Smart Routing	HTTP handling, predefined endpoints, request classification
Tier 3 — Query Orchestration and Firewall	Keyword routing, AI SQL generation, five-layer validation
Tier 4 — Databricks Unity Catalog	Secure views, row-level security, column masking, raw data storage
Tier 5 — Post-Execution Governance	Masked column detection, analytical dependency checks, AI summary control

A request enters at Tier 1, flows downward through Tiers 2 and 3, reaches Databricks at Tier 4, and the results pass through Tier 5 before being returned to the user. Security is enforced independently at Tiers 3, 4, and 5 — any single tier can block unauthorized access on its own.

---

3. Tier 1 — End Users and Identity

Every request begins with a user interacting with the web dashboard. The dashboard presents a role selector (admin, manager, finance_user, auditor, viewer) and, for finance users, a company selector.

When a request is sent to the API, the selected role and company are transmitted via HTTP headers (`X-User-Role`, `X-User-Company`). These headers drive application-level RBAC enforcement in `rbac.py`.

At the Databricks level, identity is resolved differently. The Flask API connects to Databricks using a single personal access token (PAT). Databricks associates this token with the account that generated it and exposes the account email through the `current_user()` SQL function. The secure views use this email to look up the user's role and company assignment in the `governance_schema.users` table.

This creates a dual-layer identity model. In production, each user would authenticate with their own token, and `current_user()` would resolve to their individual identity. In this POC, the application-level RBAC layer (`rbac.py`) mirrors the database rules and applies them based on the HTTP headers, compensating for the single-token limitation.

---

4. Tier 2 — API Gateway and Smart Routing

The Flask application (`app.py`) exposes eleven endpoints. Ten are predefined GET endpoints that map directly to hardcoded SQL queries. The eleventh is a POST endpoint (`/api/query-router`) that accepts natural language questions.

Predefined endpoints include yearly revenue, quarterly revenue, company-specific breakdowns, top-N rankings by profit/revenue/cost, period filters, and a platform summary. Each endpoint calls a shared response helper (`_respond()`) that executes the corresponding SQL from `predefined_queries.py`, applies RBAC filtering via `rbac.py`, and returns a standardized JSON response.

The query router is the entry point for all natural language interactions. It first attempts to match the question against keyword patterns (described in Tier 3). If no pattern matches and a Groq API key is configured, it delegates to the AI engine. If no API key is available, it returns a stub response with suggested predefined queries.

All database communication passes through `db.py`, which manages Databricks connections using credentials from the `.env` file. The connection uses TLS with certificate validation via the `certifi` library.

Key design decision: The API is intentionally a thin proxy. It does not filter rows, mask columns, or enforce access rules in its core query path. That responsibility belongs to the database (Tier 4) and the post-execution RBAC layer. This means the API has no "god mode" — it cannot return data that the database has already excluded.

---

5. Tier 3 — Query Orchestration and Firewall

This tier handles the transformation of user intent into validated SQL. It has two paths.

Path A — Keyword Router (Predefined Route)

The keyword router (`route_question()`) receives the user's natural language input and attempts to match it against a priority-ordered list of patterns. It detects company IDs via regex (`C001`, `C002`, etc.), year references (`2024`, `2023`), quarter references (`Q1`, `Q3`), and intent keywords (profit, revenue, cost, summary).

If a match is found, the router returns the corresponding predefined query key and any extracted parameters. The query is then executed through the same safe, hardcoded SQL used by the GET endpoints.

For finance users, questions containing possessive language ("our", "my", "we") bypass the keyword router entirely and are forwarded to the AI engine. This is a deliberate design choice — the keyword router's predefined queries do not dynamically inject company-scoped WHERE clauses, so the AI engine is needed to produce correctly filtered SQL for these cases.

Path B — AI Engine (AI-Powered Route)

When the keyword router cannot resolve a question, the AI engine takes over. This involves four sequential stages:

Stage 1 — Dynamic System Prompt Construction. The function `_build_system_prompt(user_role, company_id)` generates a per-request prompt that includes the user's exact role, their company restriction (if applicable), all available company IDs and names, the column schemas of both secure views, and strict generation rules. For finance users, the prompt contains a mandatory directive to include `WHERE company_id = '{cid}'` in every generated query. This prevents the AI from producing unscoped queries that would return cross-tenant data.

Stage 2 — SQL Generation. The prompt and user question are sent to the Groq API (model: `llama-3.3-70b-versatile`, temperature: 0.1). The AI returns a raw SQL string. The low temperature setting minimizes creative variation and produces more deterministic output.

Stage 3 — Five-Layer SQL Firewall. Every AI-generated query passes through five sequential validation checks before execution:

- Layer 1 (Statement Check): Rejects anything that does not begin with SELECT.
- Layer 2 (Keyword Block): Rejects queries containing DROP, DELETE, UPDATE, INSERT, ALTER, CREATE, TRUNCATE, UNION, GRANT, REVOKE, or any other mutation/escalation keyword.
- Layer 3 (Target Enforcement): Rejects queries that do not reference one of the two permitted secure views (`secure_revenue_yearly` or `secure_revenue_quarterly`) by fully-qualified name.
- Layer 4 (Schema Block): Rejects queries that reference protected objects including `revenue_transactions`, `audit_logs`, `users`, or the `governance_schema` namespace.
- Layer 5 (Stack Prevention): Rejects queries containing semicolons followed by additional statements, preventing multi-statement injection.

If any layer fails, the query is discarded and an error response is returned. The query never reaches Databricks.

Stage 4 — LIMIT Injection. If the validated query does not already contain a LIMIT clause, one is appended based on the user's role: 100 for admin/manager, 50 for finance_user/auditor, 20 for viewer.

6. Tier 4 — Databricks Unity Catalog

This is the core of the governance architecture. All financial data, aggregation logic, identity mappings, and access policies reside here.

6.1 Catalog and Schema Structure

The platform uses a single Unity Catalog (`governed_platform_catalog`) with two schemas:

- `finance_schema` — contains all financial data: the raw transactions table, aggregation views, and the two secure views that are the only externally accessible objects.
- `governance_schema` — contains the users table (role and company mappings) and the audit_logs table. This schema is never exposed to end users or the API.

The separation ensures that business data and security configuration are independently managed and permissioned.

6.2 Data Layer (Raw Transactions)

The `revenue_transactions` table stores individual financial events: transaction ID, company ID, transaction date, revenue type (Product, Service, Subscription), region, revenue amount, cost amount, profit amount, and currency. The current dataset contains 40 transactions across five companies (8 per company) spanning 2023 and 2024.

This table has no access grants. It cannot be queried directly by any user or service account.

6.3 Aggregation Layer (Business Views)

Two views aggregate the raw transactions:

- `revenue_yearly_view` groups by company_id and year, producing total_revenue, total_cost, and total_profit per company per year.
- `revenue_quarterly_view` adds a quarter dimension to the same aggregation.

These views contain no security logic. They perform pure mathematical aggregation. They also have no access grants and cannot be queried directly.

6.4 Policy Enforcement Layer (Secure Views)

The two secure views (`secure_revenue_yearly` and `secure_revenue_quarterly`) are the only objects accessible to the API. They combine the aggregation views with the users table to enforce access policy at query time.

Row-Level Security (RLS) is implemented in the WHERE clause. The view joins the aggregation data with the users table on `u.email = current_user()`. The WHERE clause then applies role-based filtering:

- Admin, manager, auditor, and viewer roles pass all rows (all companies visible).
- Finance_user role passes only rows where `v.company_id = u.company_id` (single-company isolation).

Column-Level Security (CLS) is implemented using CASE expressions in the SELECT clause:

- `total_cost` is set to NULL when the role is viewer.
- `total_profit` is set to NULL when the role is auditor or viewer.
- All other roles receive the actual values.

The resulting visibility matrix:

Column	Admin	Manager	Finance User	Auditor	Viewer
company_id	All	All	Own company only	All	All
total_revenue	Visible	Visible	Visible	Visible	Visible
total_cost	Visible	Visible	Visible	Visible	NULL
total_profit	Visible	Visible	Visible	NULL	NULL

6.5 Permission Lockdown

After schema creation, all privileges are revoked on both schemas. SELECT is then granted exclusively on the two secure views. This means that even with valid Databricks credentials, a user cannot query raw transactions, aggregation views, the users table, or audit logs directly. The secure views are the sole data access path.

6.6 Audit Logging

The `audit_logs` table records every data access event: the requesting user's email, their role at the time of access, the query type, the accessed object, the timestamp, and the company context. This supports compliance requirements (SOC 2, financial reporting standards) and enables forensic analysis of access patterns.

---

7. Tier 5 — Post-Execution Governance

After Databricks returns query results, the platform performs three additional checks before delivering data to the user.

7.1 Application-Level RBAC (`rbac.py`)

Because the POC uses a single Databricks service token, the database-level RLS and CLS operate under the token owner's identity (admin). The `rbac.py` module compensates by re-applying the same row filtering and column masking rules in Python, using the role and company ID from the HTTP request headers. This ensures

that the dashboard experience accurately reflects each role's access level, even though the underlying Databricks query executes with admin privileges.

In a production deployment where each user authenticates with their own token, this layer would serve as a redundant safety net rather than the primary enforcement mechanism.

7.2 Masked Column Detection

The governance awareness layer inspects the returned data for evidence of column masking. If the `total_profit` column contains only NULL values across all returned rows, the system flags it as masked. Similarly for `total_cost`.

7.3 Analytical Dependency Check

If masked columns are detected, the system evaluates whether the user's original question depends on the masked data. For example, an auditor asking "which company had the highest profit margin" requires profit data — which is masked for auditors. In this case, the governance layer sets the status to "limited" and suppresses the AI summary entirely, replacing it with an access denial message. This prevents the AI from generating misleading analysis based on NULL values.

If the question does not depend on the masked columns (e.g., an auditor asking about revenue trends), the data is returned normally with masked columns displayed as dashes in the UI.

7.4 AI Summary Generation

When the governance status is "ok", the results are passed to a secondary Groq model (`llama-3.1-8b-instant`) for natural language summarization. The summary prompt includes company context for finance users, ensuring the AI refers to the specific company by name rather than making platform-wide generalizations.

---

8. Security Model — Defense in Depth

The platform enforces security at three independent points. A failure at any single point does not compromise data confidentiality.

Point 1 — SQL Firewall (Tier 3). Prevents execution of unsafe or unauthorized SQL before it reaches the database. Blocks mutations, schema access, and multi-statement injection.

Point 2 — Secure Views (Tier 4). Enforces row-level and column-level access control at the database engine level, based on the authenticated identity. This operates independently of the application and cannot be bypassed by API manipulation.

Point 3 — Post-Execution Governance (Tier 5). Detects masked columns in returned data and prevents the AI from generating analysis that would circumvent column-level restrictions.

Additionally, predefined queries never interpolate user input into SQL strings, eliminating SQL injection as a vector. The HTTP role header (`X-User-Role`) does not influence database-level security — it only affects the application RBAC layer and AI prompt construction.

9. Technology Stack

Component	Technology
Database Platform	Databricks Unity Catalog (Azure)
Query Engine	Databricks SQL Warehouse
API Framework	Python / Flask
AI Model (SQL Generation)	Groq — Llama 3.3 70B Versatile
AI Model (Summarization)	Groq — Llama 3.1 8B Instant
Frontend	HTML / CSS / JavaScript (single-page)
Authentication	Databricks Personal Access Token
Configuration	Environment variables (`.env`)

10. Limitations and Production Considerations

Single service token. The current implementation uses one Databricks PAT for all API requests. In production, each user should authenticate independently so that `current_user()` resolves to their actual identity.

Static user table. Role assignments are managed by direct SQL inserts. A production system would require a user management interface with approval workflows.

No real-time data ingestion. The POC operates on a static dataset of 40 transactions. Production deployment would require integration with data ingestion pipelines.

AI model dependency. The natural language interface depends on Groq API availability. The system degrades gracefully — keyword routing and predefined endpoints continue to function without AI.

Audit log insertion. The current implementation logs access events from the application layer. A production system should also capture Databricks-native query logs for comprehensive coverage.

End of Document